

FIT5225 2026 S1 Assignment 2(group) Marking Rubric

Grading Scale:

- **100% (Fully Implemented):** The feature is fully functional and meets all specifications without errors.
- **50% (Partially Implemented):** Feature is present but has bugs, missing edge cases, or only partially meets the requirements.
- **0% (Not Implemented):** Feature is missing, not working at all, or fails completely during testing.

Category & Criterion	Max Mark	100% (Fully Implemented)	50% (Partially Implemented)	0% (Not Implemented)
1. Authentication and Authorisation (10%)				
1.1 Sign-up, Sign-in, Sign-out	4	AWS Cognito fully handles registration (with email, first/last name, password), email verification, login, and sign-out securely.	Cognito is used, but some features (e.g., email verification or specific user details) are missing or bugged.	Not implemented or fails to authenticate users.
1.2 Access Control & Redirection	3	Unauthenticated access is completely blocked for all endpoints (except sign-up); users are successfully redirected.	Some endpoints are protected, but others are exposed, or redirection is poorly handled.	No protection; endpoints are publicly accessible.
1.3 Fine-Grained Permissions (IAM)	3	Uploads, DB access, and Lambda invocations are properly restricted using precise IAM roles/policies across clouds.	Basic IAM roles exist but are overly permissive or lack proper cross-cloud token handling.	Missing IAM roles; permissions are broken or nonexistent.
2. Core Functionalities (50%)				

2.1 File Handling (20 marks)				
2.1.1 File Upload & Deduplication	6	Files upload successfully via API/SDK; the system reliably prevents duplicate uploads using a file checksum.	Upload works, but deduplication fails or relies on basic file names instead of checksums.	Upload fails, or deduplication is not implemented.
2.1.2 Thumbnails & Video Processing	6	Triggers thumbnail creation for images (aspect ratio maintained/compressed) and extracts 1 frame/sec for videos.	Thumbnail or video processing works, but ignores aspect ratio, compression, or extracts wrong frame rates.	Processing fails, or video/image extraction is not implemented.
2.1.3 ML Tagging & DB Insertion	8	Serverless function successfully detects species using the ML model, saves file type, tags, and URLs (image/thumbnail) to the DB.	ML detection or DB insertion works, but misses data (e.g., missing URLs or file types) or requires manual triggers.	ML tagging or DB insertion completely fails.
2.2 Queries (20 marks)				
2.2.1 Find by Tags (Counts/Species)	6	JSON API successfully handles logical AND operations for multiple tags with minimum counts, and simple species queries, returning correct URLs.	Query works but fails on logical AND (does OR instead), ignores minimum counts, or returns broken URLs.	Tag/Species query is not implemented or completely broken.
2.2.2 Find by Thumbnail URL	6	API accurately maps a provided thumbnail URL to its corresponding	Partially works but returns incorrect mappings or fails on specific image	Not implemented or fails to return full

		full-size image URL.	types.	URLs.
2.2.3 Find by Uploaded File	8	API successfully accepts a file, detects tags, and returns matching database URLs <i>without</i> permanently storing the query file.	Works, but permanently stores the query file, or tag detection for the query file is inaccurate.	Not implemented or fails to process the uploaded file for querying.
2.3 Data Management & Notifications (10 marks)				
2.3.1 Manual Tag Edit (Bulk)	3	POST API successfully allows bulk addition/removal of tags using operations (1=add, 0=remove) and ignores unassigned tags on deletion.	API works for single files but fails on bulk, or fails to properly handle the add/remove binary logic.	Tag editing is not implemented or fails.
2.3.2 Delete Files	3	API correctly removes full images, thumbnails, and videos from cloud storage AND removes the corresponding database entries.	Deletes from DB but leaves files in storage (or vice versa), or fails on bulk deletions.	Deletion not implemented.
2.3.3 Tag-Based Notifications	4	AWS SNS (or equivalent) successfully triggers and sends email notifications when specific watched tags are added to the system.	Notifications trigger but are inconsistent, lack specific tag filtering, or email formatting is broken.	Notifications not implemented.
3. User Interface (20%)*				

3.1 Auth & Upload UI	6	Clean, functional UI for sign-in, sign-up, and media uploading (including clear feedback on deduplication/success).	UI exists but is clunky, lacks error handling, or only has partial functionality support.	No UI provided for these features.
3.2 Queries UI	6	UI allows complex queries, visually previews thumbnails, and allows users to click thumbnails for full-size images.	Query UI works but lacks thumbnail previews, or clicking does not retrieve the full-size image.	No UI provided for query functionalities.
3.3 Tag Edit & General Usability	8	UI successfully supports bulk tag editing and file deletion. Professional, user-friendly design.	UI supports some tag management but lacks bulk support or is highly unintuitive.	No UI provided for tag/data management.
4. Demo and Final Reports (20%)				
4.1 Demo Performance (15 marks)				
4.1.1 Architecture Presentation	5	Clear, concise 3-5 minute explanation of the multi-cloud architecture and design choices. All members present.	Rushed/unclear presentation, missing architectural details, or not all members are actively involved.	No presentation or complete misunderstanding of own architecture.
4.1.2 Application Demo & Q&A	10	Smooth, error-free demonstration of all core features. Team successfully answers examiner Q&A confidently.	Demo has noticeable bugs/crashes, or team struggles significantly to answer Q&A.	Demo fails to show core functionality.

4.2 Final Reports (5 marks)				
4.2.1 Team Report	3	Includes an official multi-cloud architecture diagram, contribution table, user guide, and a shared Git repository link with commits from all members.	Missing one or two elements (e.g., unofficial icons, no user guide, or unbalanced Git commits).	Report not submitted or missing majority of requirements.
4.2.2 Individual Report	2	Clear description of personal role and a thoughtful 150-word reflection on teamwork and challenges.	Missing either the role description or the reflection, or lacks sufficient detail.	Not submitted.
Total Marks	100			

Additional Notes:

- 1) *As per the assignment specifications, if a team opts not to create a UI, the maximum mark they can receive for the UI section is capped at D (14/20), provided alternative demonstration methods (such as CLI , scripts or demo through postman) are flawless. If no UI and no alternative scripts are provided, the score for Section 3 will be 0.*
- 2) *At the end of the session, teams will conduct peer assessments of their fellow members' contributions and involvement in the project. Those who do not actively contribute to the group project may be **penalised by up to 50% off their group mark.***